

软件工程与实践课程项目

体系结构设计文档 (SAD)

项目名称：Mini Blog

小组成员	姓名	学号
开发人员	刘俊汝	202300172027

2026 年 6 月

目录

1	引言	3
1.1	标识	3
1.2	系统概述	3
1.3	文档概述	3
1.4	基线	3
2	引用文件	4
3	CSCI 级设计决策	4
3.1	体系结构风格选型	4
3.2	技术选型决策	5
3.3	安全设计决策	5
4	CSCI 体系结构设计	6
4.1	体系结构	6
4.1.1	程序 (模块) 划分	6
4.1.2	程序 (模块) 层次结构关系	7
4.2	全局数据结构说明	8
4.2.1	数据库设计	8
4.2.2	常量	8
4.2.3	变量	9
4.2.4	数据结构	9
4.3	CSCI 部件	10
4.3.1	部件标识与静态关系	10
4.3.2	关键类设计	11
4.4	执行概念	11
4.4.1	请求处理流程	11
4.4.2	Django 内部处理流程	12
4.4.3	并发处理	12
4.4.4	异常处理	12
4.5	接口设计	13
4.5.1	接口标识与接口图	13
4.5.2	模块间接口约定	14
5	部署设计	14
5.1	部署视图	14
5.2	部署步骤概要	15

目录	2
6 需求的可追踪性	15
6.1 从 SAD 部件到 SRS 需求的可追踪性	15
6.2 从 SRS 需求到 SAD 部件的可追踪性	16
7 注解	16
7.1 术语与定义	16
7.2 缩略语	16

1 引言

1.1 标识

- 标识号: MiniBlog-SAD-2026-001
- 标题: Mini Blog 博客网站系统——体系结构设计文档 (SAD)
- 缩略词语: SAD (Software Architecture Document); CSCI (Computer Software Configuration Item); MVC (Model-View-Controller); MTV (Model-Template-View); ORM (Object-Relational Mapping)
- 版本号: V1.0
- 开发单位: 山东大学计算机科学与技术学院
- 适用系统: Mini Blog 博客网站系统

1.2 系统概述

Mini Blog 是一个基于 B/S 架构的轻量级博客网站系统。系统采用 Python Django 框架作为后端核心, MySQL 8.0 作为数据库管理系统, 前端采用原生 HTML5、CSS3 及 JavaScript 技术构建响应式界面。系统涉及注册用户与系统管理员两种角色, 涵盖用户认证、文章管理、评论互动及后台管理等核心功能。

本系统为软件工程与实践课程的实践项目, 以个人开发形式完成。系统的详细功能需求参见《软件需求规格说明 (SRS)》, 可行性论证参见《可行性分析报告 (FAR)》。

1.3 文档概述

本文档是 Mini Blog 系统的体系结构设计文档 (Software Architecture Document, SAD), 依据 GB/T 8567-2006 《计算机软件文档编制规范》中“软件 (结构) 设计说明 (SDD)”的格式编写, 同时参考了 IEEE 42010-2022 体系结构描述标准。

文档描述系统的 CSCI 级设计决策、体系结构风格选型、模块划分与层次关系、全局数据结构、执行流程、接口设计及部署方案, 为后续的详细设计与编码实现提供架构基础。

本文档不含保密性限制内容, 可在教学范围内分发使用。

1.4 基线

本 SAD 文档基于以下基线编写:

- 《可行性分析报告 (FAR)》V1.0, 2026 年 4 月

- 《软件需求规格说明（SRS）》V1.0，2026 年 5 月
- 课程项目课题说明

2 引用文件

1. GB/T 8567-2006 计算机软件文档编制规范——软件（结构）设计说明 (SDD)
2. IEEE 42010-2022 Systems and Software Engineering — Architecture Description
3. Mini Blog 可行性分析报告（FAR），V1.0，2026 年 4 月
4. Mini Blog 软件需求规格说明（SRS），V1.0，2026 年 5 月
5. Django 4.2 官方文档，Django Software Foundation，2024
6. MySQL 8.0 Reference Manual，Oracle Corporation，2024

3 CSCI 级设计决策

3.1 体系结构风格选型

Mini Blog 系统综合采用以下体系结构风格，以平衡系统的可维护性、可扩展性与开发效率：

1. B/S 风格（Browser/Server）

- 客户端仅需标准 Web 浏览器，无需安装专用软件
- 服务端集中部署与维护，用户端免更新
- 天然支持跨平台访问（Windows、macOS、Linux、移动端）

2. 层次结构风格

- 系统划分为表示层、业务逻辑层、数据持久层三层
- 每层仅依赖其直接下层，上层不感知下层实现细节
- 支持逐级抽象与独立演化

3. MTV 模式（Django 的 MVC 变体）

- Model：数据模型层，通过 ORM 映射关系数据库表
- Template：表现层，HTML 模板负责页面渲染
- View：业务逻辑层，处理 HTTP 请求，协调 Model 与 Template

体系结构风格的组合关系如图1所示。

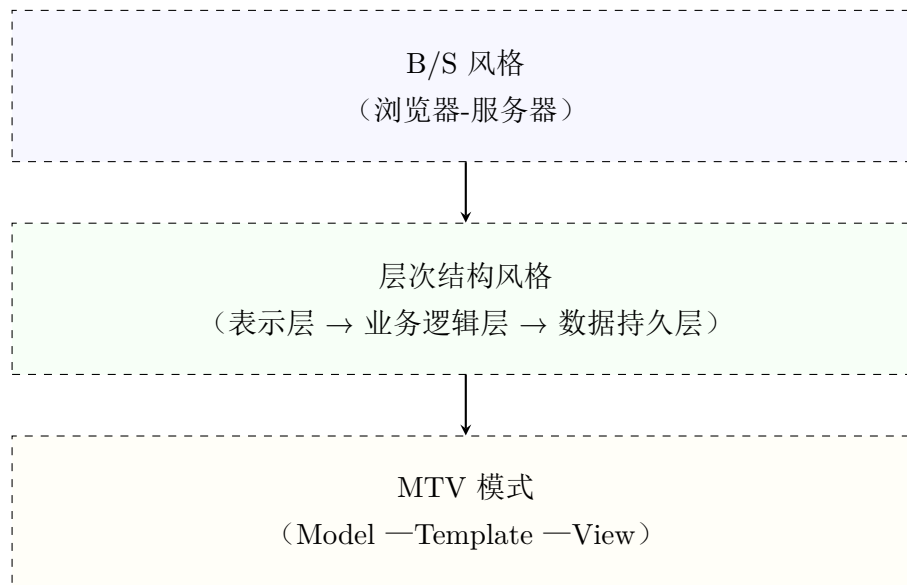


图 1: 体系结构风格组合

3.2 技术选型决策

层次	选型	决策理由
Web 服务器	Nginx 1.24+	高性能反向代理，静态资源服务，负载均衡
WSGI 容器	Gunicorn 21+	成熟稳定，与 Django 配合良好，配置简洁
Web 框架	Django 4.2	内置 ORM、Auth、Admin，开发效率高
数据库	MySQL 8.0	关系型数据库，课程要求，生态成熟
前端技术	HTML5/CSS3/JS	原生技术，无重型框架依赖，学习曲线低
会话管理	Django Session	服务端会话，CSRF 保护内置
密码存储	PBKDF2+Salt	Django 默认方案，迭代 390000 次

表 1: 技术选型决策表

3.3 安全设计决策

- **认证:** 基于 Django Authentication 中间件，Session 维持登录状态，30 分钟超时
- **授权:** 视图级权限校验——普通视图允许匿名访问，写操作视图要求登录，管理视图要求管理员角色
- **CSRF 防护:** Django CSRF 中间件对所有 POST/PUT/DELETE 请求验证 Token
- **XSS 防护:** Django 模板引擎自动转义输出内容；用户输入存储前经 HTML 标签过滤

- **SQL 注入防护**: 全程使用 Django ORM 参数化查询, 不使用原始 SQL 拼接
- **密码安全**: PBKDF2+SHA256 哈希加盐, 不存储明文

4 CSCI 体系结构设计

4.1 体系结构

4.1.1 程序 (模块) 划分

Mini Blog 系统依据 Django 的项目组织惯例与功能内聚原则, 划分为以下顶层模块 (Django App), 如表2所示。

编号	模块名称	Django App	功能说明
M01	用户模块	accounts	用户注册、登录、登出、密码修改、个人信息管理
M02	博客模块	blog	文章的发表、修改、删除、列表展示、详情查看
M03	评论模块	comments	评论的提交、展示、删除
M04	管理模块	admin_panel	基于 Django Admin 定制的后台管理界面
M05	核心配置	miniblog	项目配置 (settings.py、urls.py、wsgi.py)

表 2: 顶层模块划分

各模块内部遵循 Django MTV 模式的典型文件组织:

- `models.py` —— 数据模型定义 (Model 层)
- `views.py` —— 视图逻辑 (View 层)
- `templates/` —— HTML 模板 (Template 层)
- `forms.py` —— 表单定义与验证
- `urls.py` —— URL 路由配置
- `admin.py` —— Django Admin 注册与定制

4.1.2 程序 (模块) 层次结构关系

系统的模块层次结构如图2所示，展现了 Web 服务器 → Django 项目 → 各 App → 内部文件的层级包含关系。

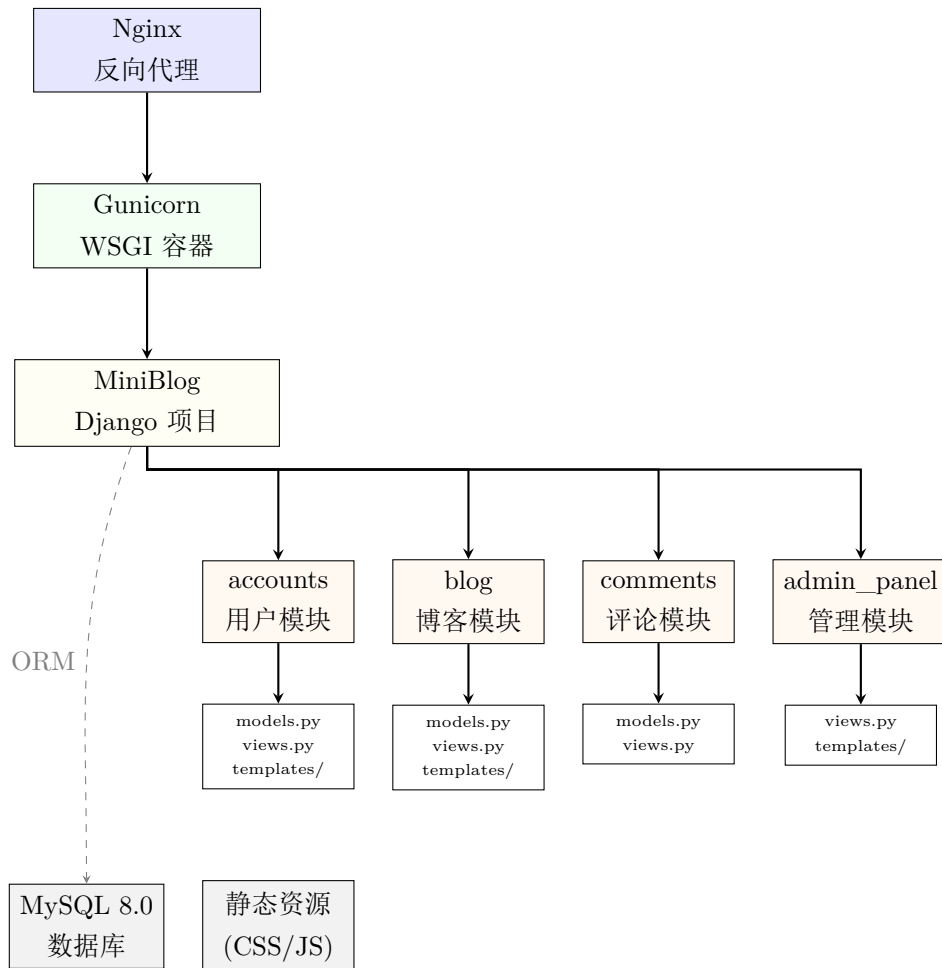


图 2: 系统模块层次结构图

调用关系方面，所有 Django App 均通过 Django 框架的内置机制进行协作：

- URL 分发器（URL Dispatcher）将 HTTP 请求路由至对应 App 的视图函数
- 视图函数通过 ORM 访问数据模型，通过模板引擎渲染页面
- App 之间通过导入 models 进行跨模块数据访问（如 blog.views 引用 comments.models）
- 中间件（Middleware）以管道方式对请求/响应进行预处理和后处理

4.2 全局数据结构说明

4.2.1 数据库设计

系统的全局数据以 MySQL 关系数据库为核心，数据模型采用 Django ORM 定义。五张核心数据表的逻辑结构如 E-R 图所示（详见 SRS 文档的图 6）。

表名	所属模块	主要字段	说明
auth_user	accounts	username, password_hash, email, date_joined, is_active	Django 内置用户表，扩展 role 字段
blog_article	blog	id, title, content, created_at, updated_at, status, author_id, category_id	文章表
comments_comment	comments	id, content, created_at, author_id, article_id	评论表
blog_category	blog	id, name, description	文章分类表
django_session	—	session_key, session_data, expire_date	Django Session 存储

表 3: 核心数据表摘要

4.2.2 常量

系统级常量集中于 Django 项目的 `settings.py` 文件，主要包括：

- SECRET_KEY: Django 加密签名密钥（生产环境应从环境变量读取）
- DEBUG: 调试模式开关（生产环境必须为 False）
- DATABASES: 数据库连接配置字典，含 ENGINE、NAME、USER、PASSWORD、HOST、PORT 键
- SESSION_COOKIE_AGE: 会话超时时间（默认 1800 秒 = 30 分钟）
- CSRF_COOKIE_HTTPONLY: CSRF Cookie HttpOnly 标志（设为 True）
- STATIC_ROOT / STATIC_URL: 静态文件收集目录与 URL 前缀
- LOGIN_URL: 登录页面 URL（默认 /accounts/login/）
- AUTH_USER_MODEL: 自定义用户模型引用

4.2.3 变量

系统运行时的全局变量由 Django 框架管理，应用层不定义 C 语言意义上的全局变量。需要说明的全局性可变状态包括：

- **Session 存储：**Django Session 引擎（默认数据库后端）维护每个已认证用户的会话状态字典，包含用户 ID、权限标识、过期时间等键值对。会话数据在 `django_session` 表中以序列化形式存储。
- **Cache（可选）：**系统可配置 Django 缓存框架（如 Memcached 或 Redis 后端），用于缓存频繁查询的文章列表、分类列表等只读数据，减少数据库压力。当前版本暂不启用。
- **请求上下文：**每个 HTTP 请求处理周期内，Django 通过 `request` 对象传递当前用户（`request.user`）、会话（`request.session`）等线程局部变量。这些变量在请求结束时自动释放。

4.2.4 数据结构

系统的核心数据结构由 Django ORM 的 Model 类定义，对应 MySQL 数据库中的关系表。以下列出四个自定义 Model 的正式定义（Django 内置的 User、Session 等表不重复列出）。

字段名	类型	约束	说明
id	AutoField	PK	自增主键
title	CharField(200)	NOT NULL	文章标题
content	TextField	NOT NULL	文章正文（Markdown 格式）
created_at	DateTimeField	auto_now_add	创建时间戳
updated_at	DateTimeField	auto_now	最后修改时间戳
status	CharField(10)	default='published'	状态：draft / published
author	ForeignKey(User)	ON DELETE CASCADE	作者外键
category	ForeignKey(Category)	ON DELETE SET NULL, null=True	分类外键

Article（文章）

字段名	类型	约束	说明
id	AutoField	PK	自增主键
content	TextField	NOT NULL	评论内容
created_at	DateTimeField	auto_now_add	创建时间戳
author	ForeignKey(User)	ON DELETE CASCADE	评论者外键
article	ForeignKey(Article)	ON DELETE CASCADE	所属文章外键

Comment（评论）

字段名	类型	约束	说明
id	AutoField	PK	自增主键
name	CharField(50)	UNIQUE, NOT NULL	分类名称
description	CharField(200)	blank=True	分类描述

Category（分类）

User（用户，扩展 Django AbstractUser） 在 Django 内置 AbstractUser（含 username、password_hash、email、date_joined、is_active 等字段）基础上，新增：

字段名	类型	约束	说明
role	CharField(10)	default='author'	角色：author / admin
bio	CharField(500)	blank=True	个人简介
avatar	ImageField	blank=True, null=True	头像图片

各数据结构之间的关系（外键约束与级联删除策略）参见 SRS 文档的 E-R 图，本文档不再重复绘制。

4.3 CSCI 部件

4.3.1 部件标识与静态关系

Mini Blog 系统的 CSCI 部件（软件配置项）以 Django 项目结构组织，各部件间的静态关系（组成与依赖）如图3所示。

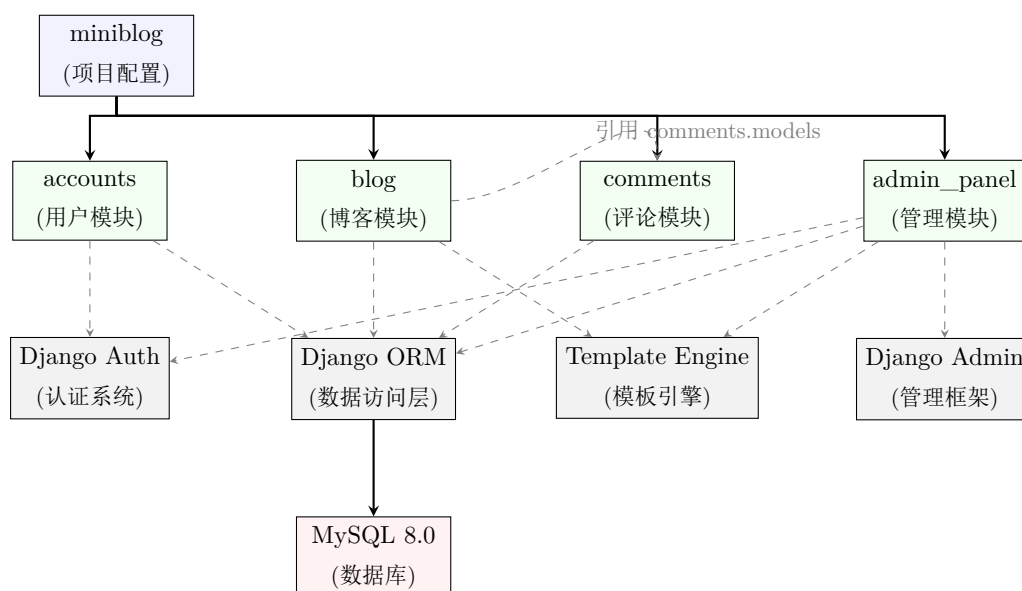


图 3: 系统组件图——部件静态关系

4.3.2 关键类设计

系统核心模型类的 UML 类图（详见 SRS 文档图 4）定义了以下关键实体及其关系：

- **User**(抽象基类): Django 内置 `AbstractUser`, 字段包括 `username`、`password_hash`、`email`、`date_joined`、`is_active`。自定义 `role` 字段区分普通用户 (author) 与管理员 (admin)。
- **Article**: 文章实体, 含 `title`、`content` (Markdown 格式)、`created_at`、`updated_at`、`status` (draft/published)。通过 `ForeignKey` 关联 `Author` (User) 和 `Category`。
- **Comment**: 评论实体, 含 `content`、`created_at`。通过 `ForeignKey` 关联 `Author` (User) 和 `Article`。
- **Category**: 分类实体, 含 `name`、`description`。与 `Article` 为一对多关系。

4.4 执行概念

4.4.1 请求处理流程

系统采用 Django 标准的请求-响应处理管线，一次典型的 HTTP 请求处理流程如图4所示。

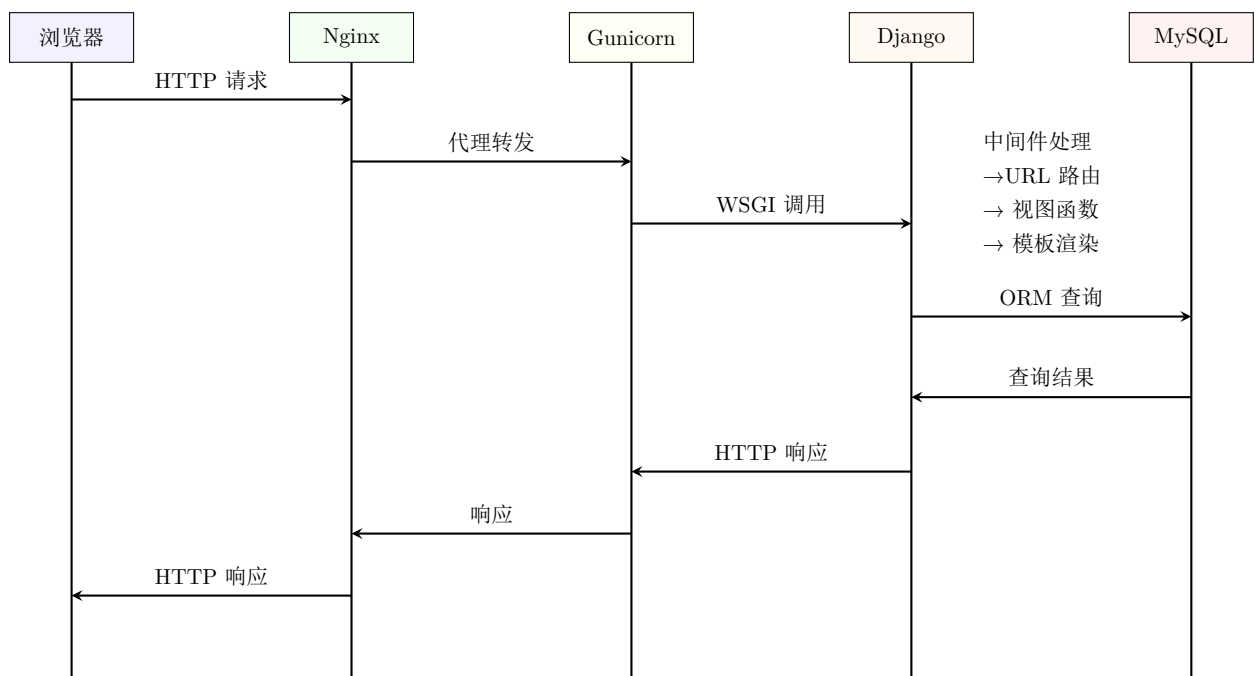


图 4: 请求-响应处理序列图

4.4.2 Django 内部处理流程

请求到达 Django 后的内部处理管线如下：

1. **中间件预处理**：请求依次经过 SecurityMiddleware、SessionMiddleware、CsrfViewMiddleware、AuthenticationMiddleware 等中间件的 process_request 方法
2. **URL 路由**：URL Dispatcher 根据 urls.py 配置将请求路径匹配到对应的视图函数
3. **视图处理**：视图函数执行业务逻辑——验证输入、查询/更新数据库、组织上下文数据
4. **模板渲染**：视图将上下文数据传递给模板引擎，渲染为 HTML 响应体
5. **中间件后处理**：响应依次经过各中间件的 process_response 方法，添加安全头等
6. **返回响应**：HTTP 响应经 Gunicorn→Nginx 送回浏览器

4.4.3 并发处理

Gunicorn 采用预分叉（pre-fork）工作模型，默认启动多个 worker 进程并行处理请求，每个 worker 独立运行 Django 应用实例。对于课程项目的并发量（< 50 用户），2-4 个 worker 进程即可满足需求。

4.4.4 异常处理

系统在运行时的异常处理分为三个层次：框架层异常、应用层异常、基础设施层异常。

1. 框架层异常（Django 内置处理）

Django 框架对常见 Web 异常提供统一处理机制：

- **404 Not Found**：URL 分发器未匹配到路由时，Django 抛出 Http404 异常。开发模式下显示调试页面，生产模式下渲染自定义 404.html 模板。
- **403 Forbidden**：CSRF 验证失败或权限不足时，Django 返回 403 响应。CSRF 失败渲染 403_csrf.html 模板，提示用户刷新页面后重试。
- **500 Internal Server Error**：视图代码抛出未捕获异常时，Django 的异常中间件捕获后返回 500 响应。生产模式下渲染 500.html 模板，隐藏堆栈细节；同时通过日志记录完整异常信息供排查。
- **302 Redirect**：未登录用户访问需认证的视图时，LoginRequiredMixin 或 @login_required 装饰器自动重定向至登录页面，URL 附带 ?next= 参数以便登录后跳回原页面。

2. 应用层异常（视图内显式处理）

业务逻辑中的预期异常在视图函数中显式捕获和处理：

- **表单验证失败：**用户提交的数据不合法（如用户名已存在、密码太短、标题为空），Django Form 的 `is_valid()` 方法返回 `False`，视图将错误信息字典传回模板，在原页面高亮显示错误字段，并保留用户已填写的合法内容。
- **权限校验失败：**用户尝试修改或删除非本人的文章/评论时，视图首先检查 `request.user == object.author`，不匹配则返回 403 并显示“您没有权限执行此操作”。
- **资源不存在：**通过 `get_object_or_404()` 快捷函数查询文章或评论，若记录不存在则自动抛出 `Http404`，避免手动处理 `DoesNotExist` 异常。

3. 基础设施层异常

- **数据库连接失败：**Django ORM 在数据库不可达时抛出 `OperationalError`。Django 自动尝试重连（取决于 `CONN_MAX_AGE` 配置）。持续失败时，所有数据库操作均失败，用户看到 500 错误页面，运维人员需检查 MySQL 服务状态。
- **事务回滚：**Django 的 `ATOMIC_REQUESTS` 数据库配置（设为 `True` 时）确保每个 HTTP 请求内的所有数据库写操作封装在同一事务中。任何操作失败时事务自动回滚，保证数据一致性。
- **静态文件缺失：**Nginx 配置中若静态文件路径不存在或未执行 `collectstatic`，浏览器端表现为 CSS/JS 加载失败（页面样式丢失）。通过部署检查清单规避此问题。

4.5 接口设计

4.5.1 接口标识与接口图

系统的主要内部与外部接口如图5所示。

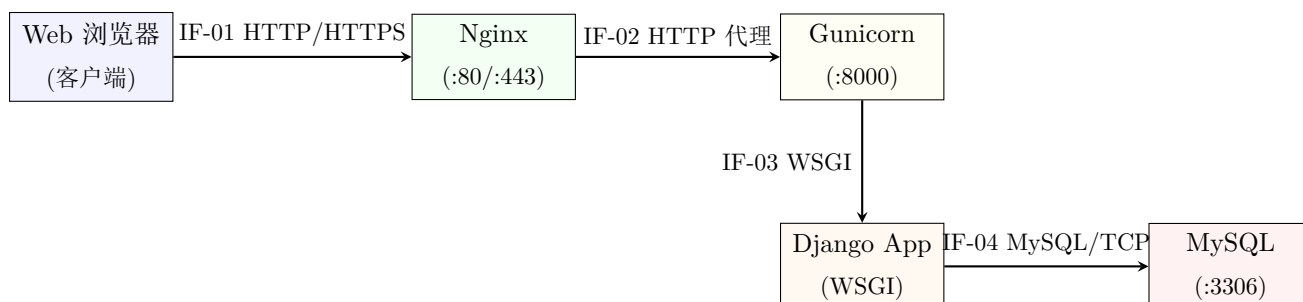


图 5: 系统接口图

各接口的详细说明如下：

编号	接口名称	协议/格式	说明
IF-01	浏览器-Nginx	HTTP/1.1, HTTPS/TLS 1.2+	客户端发起请求, 接收 HTML/CSS/JS 响应
IF-02	Nginx-Gunicorn	HTTP 代理	Nginx 反向代理将动态请求转发至 Gunicorn
IF-03	Gunicorn-Django	WSGI (PEP 3333)	Gunicorn 通过 WSGI 协议调用 Django 应用对象
IF-04	Django-MySQL	MySQL 协议, TCP:3306	Django ORM 将 Python 操作翻译为 SQL, 经 mysqlclient 驱动发送至 MySQL

表 4: 接口规格说明

4.5.2 模块间接口约定

Django App 模块间的接口遵循 Python 模块导入机制:

- `blog.views` 可 `from comments.models import Comment` 以获取文章关联的评论
- `accounts.models` 定义的 User 模型被其他 App 通过 `settings.AUTH_USER_MODEL` 或直接导入引用
- 所有跨模块数据访问必须通过 Model 类进行, 禁止跨模块直接操作数据库表

5 部署设计

5.1 部署视图

系统的部署方案如图6所示。在课程项目背景下, 可简化为单服务器部署: Nginx、Gunicorn、MySQL 运行于同一台机器 (或开发用本机)。

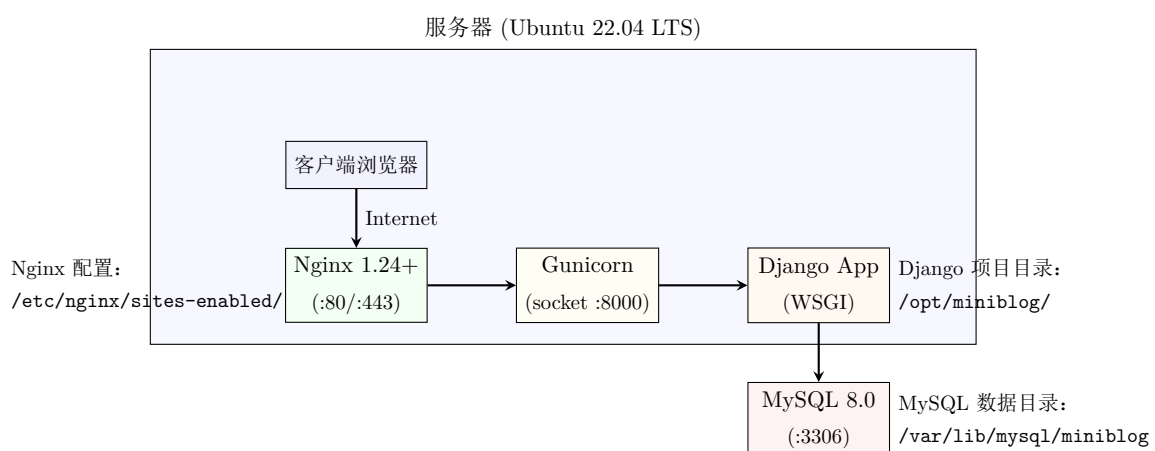


图 6: 系统部署视图

5.2 部署步骤概要

1. 安装系统依赖: Python 3.10+、MySQL 8.0、Nginx 1.24+
2. 创建 Python 虚拟环境 (conda), 安装项目依赖 (requirements.txt)
3. 配置 MySQL 数据库, 创建用户与数据库, 执行 Django migrate
4. 收集静态文件 (python manage.py collectstatic)
5. 配置 Gunicorn 服务 (systemd unit 或 Supervisor)
6. 配置 Nginx 反向代理, 指向 Gunicorn socket
7. 启用 HTTPS (可选, 使用 Let's Encrypt 证书)
8. 创建 Django 超级用户 (python manage.py createsuperuser)

6 需求的可追踪性

6.1 从 SAD 部件到 SRS 需求的可追踪性

SAD 部件	模块	覆盖的 SRS 需求
accounts.models (User)	M01 用户模块	UC01 用户注册、UC02 用户登录、UC11 管理个人信息
accounts.views	M01 用户模块	UC01、UC02、UC11
blog.models (Article, Category)	M02 博客模块	UC03 发表文章、UC06 浏览文章
blog.views	M02 博客模块	UC03、UC04 修改文章、UC05 删除文章、UC06
comments.models (Comment)	M03 评论模块	UC07 发表评论
comments.views	M03 评论模块	UC07
admin_panel.views	M04 管理模块	UC08 管理用户、UC09 管理文章、UC10 管理评论
Django Auth + Session	认证基础设施	UC01、UC02、NFR-安全
Django CSRF/XSS 中间件	安全基础设施	NFR-安全

表 5: SAD 部件 → SRS 需求追踪矩阵

6.2 从 SRS 需求到 SAD 部件的可追踪性

所有 SRS 功能需求（UC01–UC11）均已分配至对应的 SAD 部件（Django App 模块），非功能需求（安全、性能）由基础设施组件满足，无遗漏。

7 注解

7.1 术语与定义

术语	定义
SAD	Software Architecture Document，体系结构设计文档
CSCI	Computer Software Configuration Item，计算机软件配置项
MTV	Model-Template-View，Django 框架的 MVC 变体模式
MVC	Model-View-Controller，模型-视图-控制器模式
ORM	Object-Relational Mapping，对象关系映射
WSGI	Web Server Gateway Interface，Python Web 服务器网关接口
CSRF	Cross-Site Request Forgery，跨站请求伪造
XSS	Cross-Site Scripting，跨站脚本攻击

表 6: 术语与定义

7.2 缩略语

缩略语	全称
SAD	Software Architecture Document
SDD	Software Design Description
SRS	Software Requirements Specification
FAR	Feasibility Analysis Report
CSCI	Computer Software Configuration Item
MTV	Model-Template-View
MVC	Model-View-Controller
ORM	Object-Relational Mapping
WSGI	Web Server Gateway Interface
B/S	Browser/Server

表 7: 缩略语列表